

Database Management Systems - IT1223

Mr.M.N.M Shakoor, DPS,FAS, University of Vavuniya

Creating and Using a Database

Working with Database

- To get started on your own database, first check which databases currently exist.
 - Use the **SHOW** statement to find out which databases currently exist on the server:
 - **SHOW DATABASES ;**
 - To create a new database, issue the “**CREATE DATABASE**” command:
 - **CREATE DATABASE IT1223db ;**
 - Once the database has been successfully created, attempt to use it;
 - **USE IT1223db**
 - ✓ USE, like **QUIT**, does not require a semicolon. (You can terminate such statements with a semicolon if you like; it does no harm.) The USE statement is special in another way, too: it must be given on a single line.
 - Delete the database from the XAMPP server;
 - **DROP DATABASE IT1223db ;**
-

Data Types

For more information about MySQL data types, see [Data Types](#)

INT	You can specify a width of up to 11 digits
FLOAT(M,D)	You can define the display length (M) and the number of decimals (D). FLOAT(10,2) - 2 is the number of decimals and 10 is the total number of digits (including decimals).
DOUBLE(M,D)	You can define the display length (M) and the number of decimals (D).
DECIMAL(M,D)	Defining the display length (M) and the number of decimals (D).
DATE	A date in YYYY-MM-DD format For example, December 30th, 1973 would be stored as 1973-12-30.
DATETIME	A date and time combination in YYYY-MM-DD HH:MM:SS format For example, 3:30 in the afternoon on December 30th, 1973 would be stored as 1973-12-30 15:30:00.

TIMESTAMP	A timestamp between midnight, January 1, 1970 and sometime in 2037. This looks like the previous DATETIME format, only without the hyphens between numbers; 3:30 in the afternoon on December 30th, 1973 would be stored as 19731230153000 (YYYYMMDDHHMMSS).
TIME	Stores the time in HH:MM:SS format.
YEAR(M)	Stores a year in 2-digit or 4-digit format. If the length is specified as 2 (for example YEAR(2)), YEAR can be 1970 to 2069 (70 to 69). If the length is specified as 4, YEAR can be 1901 to 2155. The default length is 4.
CHAR(M)	A fixed-length string between 1 and 255 characters in length (for example CHAR(5)), right-padded with spaces to the specified length when stored. Defining a length is not required, but the default is 1.
VARCHAR(M)	A variable-length string between 1 and 65535 characters in length; for example VARCHAR(25). You must define a length when creating a VARCHAR field.

Creating and Using a Tables

Handling Tables - Creating Table

- Once you have selected a database, you can view all database tables:
 - **SHOW TABLES**
 - Let's create a table for storing pet details.
 - Table: pet
 - Table Fields:
 1. Name
 2. Owner
 3. Species
 4. Sex
 5. Birth Date
 6. Death Date
 - When creating a table consider the data types.
-

- To create a table, use the **CREATE TABLE** command:

```
CREATE TABLE pet (  
  name VARCHAR(20) ,  
  owner VARCHAR(20) ,  
  species VARCHAR(20) ,  
  sex CHAR(1) ,  
  birth DATE ,  
  death DATE) ;
```

Handling Tables - Creating Table

- Create Table Using Another Table
 - A copy of an existing table can also be created using CREATE TABLE.
 - The new table gets the same column definitions. All columns or specific columns can be selected.

```
CREATE TABLE petTableCopy AS  
SELECT name, owner  
FROM pet;
```

Handling Tables - Show, Describe, Delete Tables

- To verify that the table has been created:
 - **SHOW TABLES;**
 - To view a particular table structure, use the **DESCRIBE** command:
 - **DESCRIBE pet;**
 - To delete an entire table, use the **DROP TABLE** command:
 - **DROP TABLE pet;**
 - If you issue the **SHOW TABLE** Command now you will have the empty result.
 - Create the above mentioned table again.
-

Handling Tables - Truncate, Delete, Alter Tables

- To delete the data inside a table, but not the table itself, use the TRUNCATE command.
 - **TRUNCATE TABLE pet;** (Not allow us to specify the record, will delete the entire table)
 - **DELETE FROM pet;** (Allow us to delete specific record/s);
 - To add, delete, or modify columns in an existing table, use the ALTER command.
 - **ALTER TABLE pet ADD nick_name VARCHAR(10) ;**
 - **ALTER TABLE pet DROP COLUMN nick_name;**
 - **ALTER TABLE pet MODIFY COLUMN nick_name VARCHAR(12) ;**
-

Handling Tables - Insert values

- Use the **INSERT** statement to enter data into a table.
 - It is possible to write the **INSERT INTO** statement in two ways:
 1. Specify both the column names and the values to be inserted:
 - **INSERT INTO pet (name, owner, species, sex, birth, death) VALUES ('Fluffy', 'Harold', 'cat', 'f', '1993-02-04', NULL);**
 2. To adding values for all the columns of the table, no need to specify the column names in the query. But, the order of the values is in the same order as the columns in the table.
 - **INSERT INTO pet VALUES ('Fluffy', 'Harold', 'cat', 'f', '1993-02-04', '1997-04-05');**
 - It is also possible to only insert data in specific columns.
 - **INSERT INTO pet (name, owner, birth) VALUES ('Fluffy', 'Harold', '1993-02-04');**
-

Introducing NULL Values

- NULL means “a missing unknown value” and it is treated somewhat differently from other values.
 - To test for NULL, use the IS NULL and IS NOT NULL operators, as shown here:
 - `SELECT 1 IS NULL, 1 IS NOT NULL;`
 - You cannot use arithmetic comparison operators such as =, <, or <> to test for NULL
 - `SELECT 1 = NULL, 1 <> NULL, 1 < NULL, 1 > NULL;`
 - A common error when working with NULL is to assume that it is not possible to insert a zero or an empty string into a column defined as NOT NULL, but this is not the case. These are in fact values, whereas NULL means “not having a value.”
 - `SELECT 0 IS NULL, 0 IS NOT NULL, '' IS NULL, '' IS NOT NULL;`
-

Handling Tables - Insert values (NULL values)

- Now change the column named owner, which does not contain an empty value, while inserting a new record.
 - `ALTER TABLE pet MODIFY COLUMN owner VARCHAR(20) NOT NULL;`
 - Now try this query:
 - `INSERT INTO pet (name, owner, species, sex, birth, death) VALUES ('Fluffy', '', 'cat', 'f', '1993-02-04', NULL);`
 - `INSERT INTO pet (name, owner, species, sex, birth, death) VALUES ('Fluffy', NULL, 'cat', 'f', '1993-02-04', NULL);`
-

Handling Tables - Insert values (from the local text file)

- You can load the sample data as a text file as well.
 - You could create a text file '**pets.txt**' containing one record per line.
 - Values can be separated by tabs or commas, and given in the order in which the columns were listed in the **CREATE TABLE** statement.
 - Then load the data via the **LOAD DATA** Command.
-

- Use the following sample data to load it into the students table.

Name	Owner	Species	Sex	Birth	Death
Fluffy	Harold	Cat	F	04/02/1993	
Claws	Gwen	Cat	M	17/03/1994	
Buffy	Harold	Dog	F	13/05/1989	
Fang	Benny	Dog	M	27/08/1990	
Bowser	Diane	Dog	M	31/08/1979	1995/07/29
Chirpy	Gwen	Bird	F	11/09/1998	
Whistler	Gwen	Bird		09/12/1997	
Slim	Benny	Snake	M	29/04/1996	

- You could create a text file pet.txt containing one record per line, For missing values (such as unknown sexes or death dates for animals that are still living), you can use NULL values.
 - To represent these in your text file, use \N (backslash, capital-N).
 - For example, the record for Whistler the bird would look like this (where the whitespace between values is a single tab character):
 - Whistler Gwen bird \N 1997-12-09 \N
 - To load the text file pet.txt into the pet table, use this statement:
 - **LOAD DATA LOCAL INFILE '/path/pets.txt' INTO TABLE pet;**
 - **OR: LOAD DATA LOCAL INFILE 'path/pets.txt' INTO TABLE pet FIELDS TERMINATED BY '\t' LINES TERMINATED BY '\r\n';**
-

To be Continued..!
